



PROJECT PHASE 2

CSS325 Database Systems

DATASET : Football Data from Transfermarket

Team Members

- | | |
|--------------------------------------|-----------------------|
| 1. Mr. Sorravit Leeprasertsuk | ID. 6622780219 |
| 2. Mr. Panod Manodee | ID. 6622782033 |
| 3. Mr. Apichat Laohawan | ID. 6622770491 |
| 4. Mr. Krit Meesiri | ID. 6622780037 |
| 5. Mr. Charinrat Meksuwan | ID. 6622780490 |

Section 4 Group No. 7

Submitted to

Asst. Prof. Dr. Preecha Tangworakitthaworn

October 18, 2025

Semester 1/2025

School of Information, Computer and Communication Technology

**Sirindhorn International Institute of Technology
Thammasat University**

Table of Contents

Contents	Page
Introduction.....	1
Revised ERD.....	2
Mini-ERD	3
8 Steps Transformation & Diagram.....	4
3.1. 8 Steps Transformation.....	4
3.2. 8-Step Diagram	7
Data Dictionary.....	8
Database Implementation in MySQL Server.....	9
6.1 Imports of the initial Kaggle dataset.....	9
6.2 Creation of supplementary data objects.....	10
6.3 Implementation of Primary and Foreign Keys.....	11
Links	14

1. Introduction

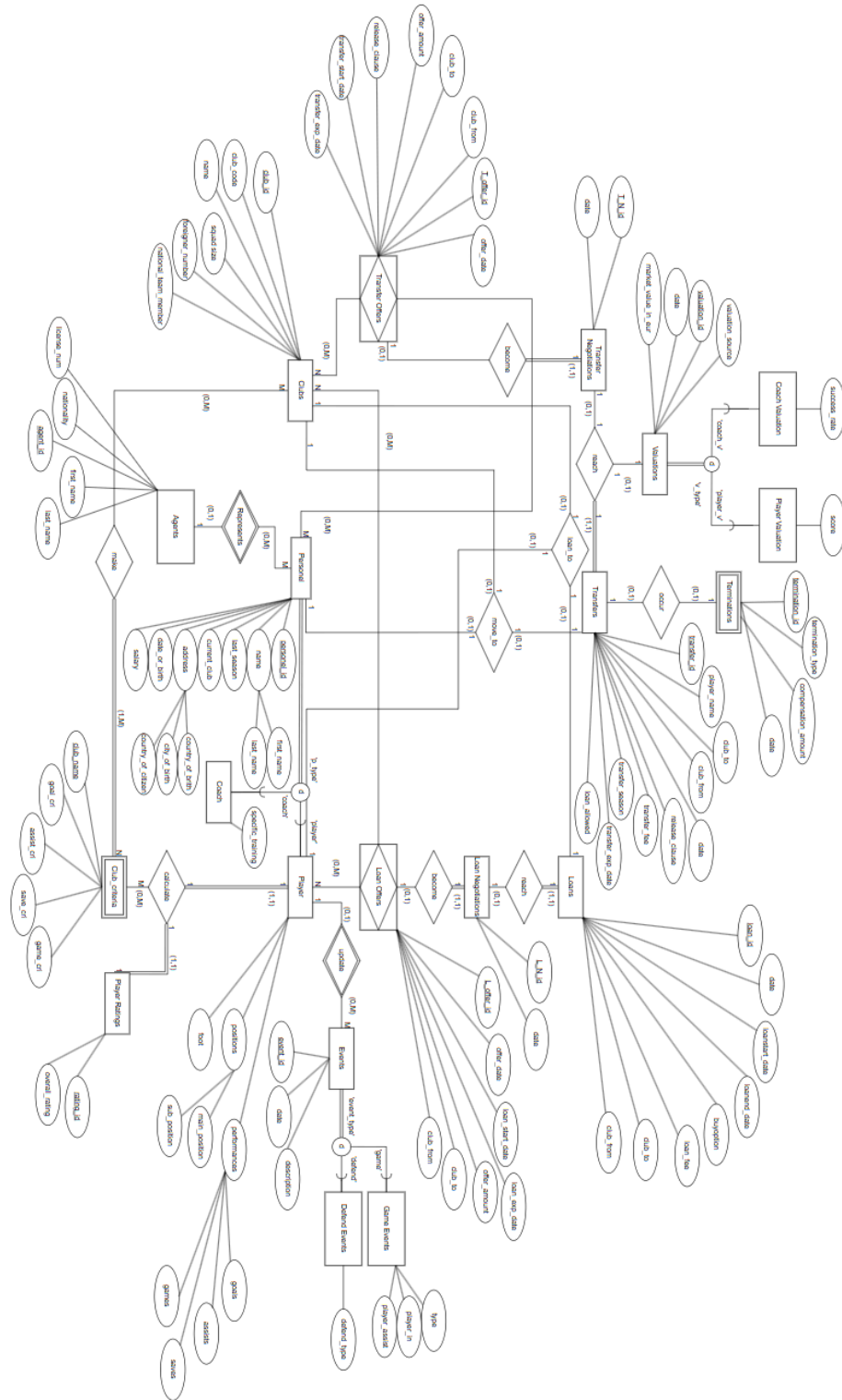
This project report is prepared as part of CSS325: Database System. The purpose of the project is to apply database design concepts in a real-world context by selecting a dataset from Kaggle and developing a conceptual database model. Our group selected a football dataset from Transfermarkt, a widely recognized platform that provides detailed information about players, clubs, and transfers. The dataset, available in .CSV format, includes both transactional data (such as appearances, player valuations, and transfers) and master data (such as player and club information), making it suitable for practicing database design that reflects real-world relationships in the football industry.

In Phase 1, we focused on the database analysis and conceptual design by identifying key entities, defining business rules, and creating an Entity Relationship Diagram (ERD) using Chen notation to represent the relationships among players, clubs, transfers, and appearances.

In Phase 2, we take on the role of database developers, implementing the conceptual model into an actual database system. The main goal of this phase is to store, manipulate, and query meaningful data based on the conceptual design developed earlier. This stage bridges the gap between database analysis and implementation, transforming the conceptual model into a functioning database that supports the management and retrieval of football-related data effectively.

2.Revised ERD

The attributes are modified to eliminate the foreign keys in the ERD and be more appropriate for each entity.

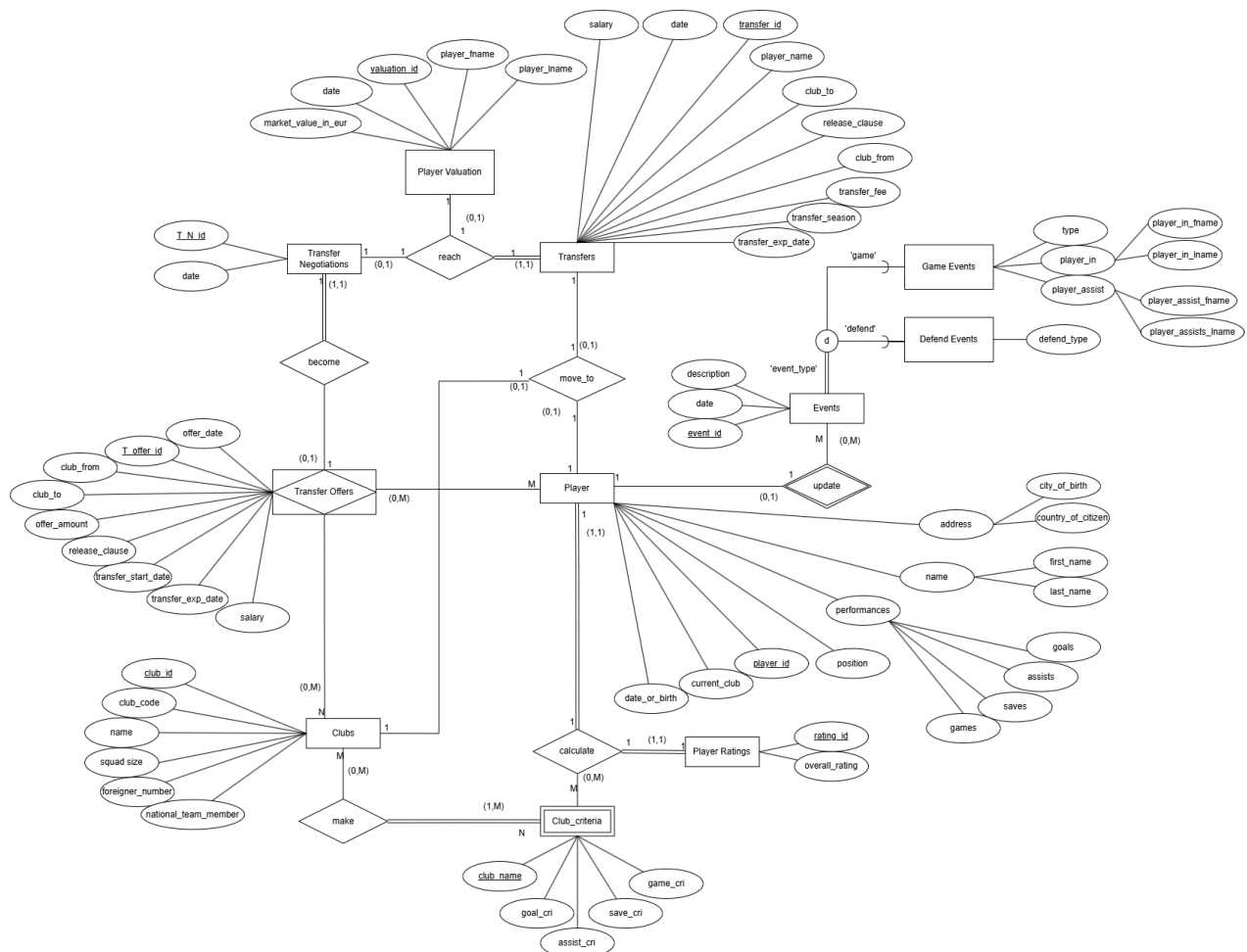


3. Mini-ERD

We selected 10 key entities from the previous ERD to represent the main components and processes of the football domain.

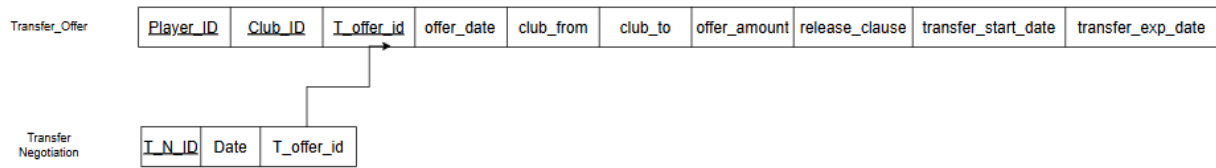
The selected entities include 6 master entities: Player, Club, Player Valuations, Events, Game Event, Defend Event; and 4 transactional entities: Club criteria, Player Ratings, Transfer, Transfer Negotiations.

All 5 of the original kaggle entities are retained: Player, Clubs, Player Valuations, Transfer, and Game Event.



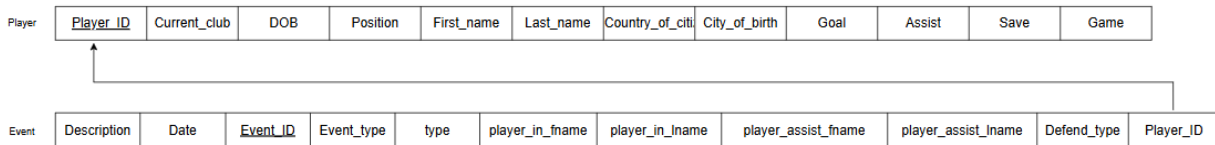
Step 3: Transforming One to One

A one to one relationship between Transfer_Offer and Transfer Negotiation was transformed by including the foreign key T_offer_id in the Transfer Negotiation table.



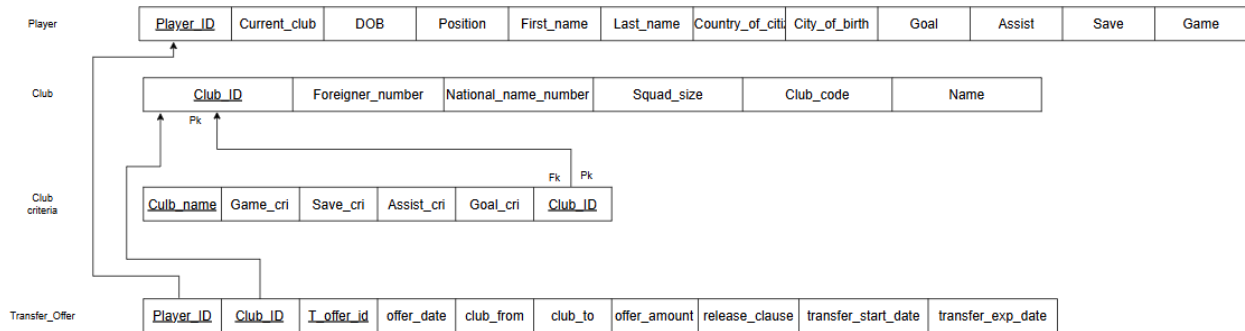
Step 4: Transforming One to Many

A player can update many events, so Player_ID was added as a foreign key in the Event table.



Step 5: Transforming Many to Many

A many-to-many relationship exists between Player and Club. It is resolved by creating the Transfer_Offer table, which links both entities using Player_ID and Club_ID as foreign keys.

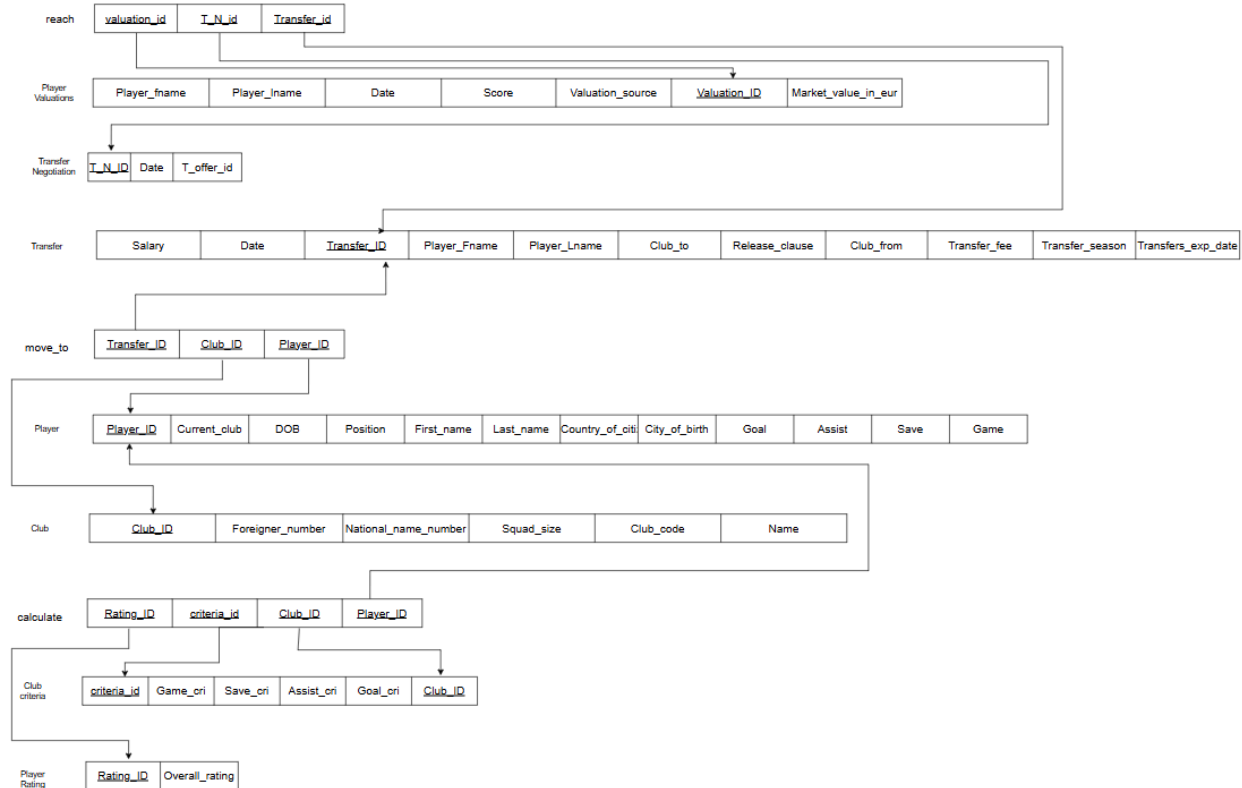


Step 6: Transforming Multi-Valued attributes

This process does not involve multiple entities.

Step 7: Transforming n-ary relationship type

The move_to table links Transfer, Club, and Player, storing their primary keys to represent the n-ary relationship.

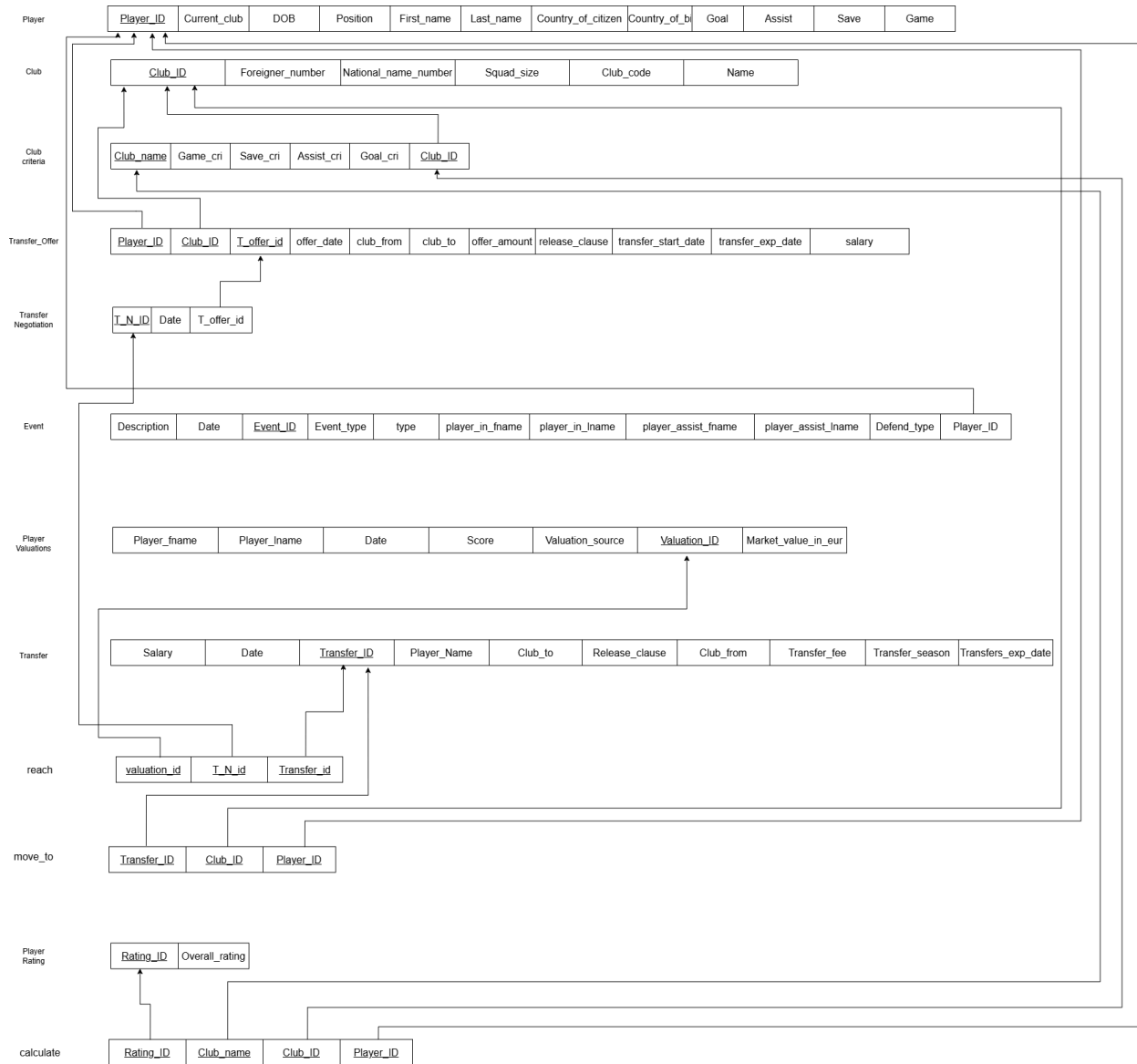


Step 8: Transforming specialization or generalization

Using case 8C for disjoint, Event has subclasses Game_Event and Defend_Event. We convert these into Event table.

Event	Description	Date	Event_ID	Event_type	type	player_in_fname	player_in_lname	player_assist_fname	player_assist_lname	Defend_type
-------	-------------	------	----------	------------	------	-----------------	-----------------	---------------------	---------------------	-------------

4.2. 8-Step ERD-to-Relational Transformation Diagram



The data dictionary has been constructed based on the relational database schema obtained from the previous chapter.

Table name	Attribute name	Contents	type	format	Nullable	Range	Key	FK referenced table		
Player	player_id	player's specific number	int	x	Y		PK			
	current_id	current club's specific number	int	x						
	DOB	player's date of birth	date	yyyy-mm-dd						
	position	player's position in a game	char(2)	XX						
	fname	player's first name	varchar(20)	Xxxxx	Y					
	lname	player's last name	varchar(20)	Xxxxx						
	country_of_citizen	player's country of citizen	varchar(20)	Xxxxx						
	country_of_birth	player's country of birth	varchar(20)	Xxxxx						
	goal	number of goals scored by a player	int	x	Y					
	assist	number of assists performed by a player	int	x	Y					
	save	number of saves performed by a player	int	x	Y					
	game	average number of games attended by a player	int	x	Y					
	Club	club_id	club's specific number	int	x			PK		
		foreigners_number	number of foreigners in the club	int	x					
national_team_players		number of players that are part of the national team	int	x						
squad_size		number of players in the club	int	x						
club_code		official abbreviation of the club's name	varchar(50)	Xxxxxx						
name		club's name	varchar(50)	Xxxxxx						
Club Criteria	criteria_id	club criteria's specific number	varchar(50)	Xxxxxx	Y	1-100	PK			
	game_cri	weighted criteria for games played by a player	int	x					Y	
	save_cri	weighted criteria for saves by a player	int	x					Y	
	assist_cri	weighted criteria for assists performed by a player	int	x					Y	
	goal_cri	weighted criteria for goals scored by a player	int	x					Y	
	club_id	club's specific number	int	x						PK,FK
Transfer Offer	player_id	player's specific number	int	x	Y		PK,FK PK,FK PK	player_id(Player) club_id(Club)		
	club_id	club's specific number	int	x						
	t_offer_id	transfer offer's specific number	int	x						
	offer_date	date of the offer	date	yyyy-mm-dd						
	club_from	name of the player's original club	varchar(50)	Xxxxxx						
	club_to	name of the club offering the contract	varchar(50)	Xxxxxx						
	offer_amount	offered amount to buy the player	int	yyyyyy						
	release_clause	offered release clause amount	int	yyyyyy						
	transfer_start_date	transfer start date	date	yyyy-mm-dd						
	transfer_exp_date	contract expiration date	date	yyyy-mm-dd						
	salary	offered salary	int	yyyyyy						
	Transfer Negotiations	t_n_id	transfer negotiation's specific number	int					x	
date		date of the negotiation	date	yyyy-mm-dd	FK	t_offer_id(Transfer Offer)				
t_offer_id		transfer offer's specific number	int	x						
Events	date	date of the event	date	yyyy-mm-dd	Y Y Y Y		PK			
	event_id	event's specific number	int	x						
	event_type	type of the event (game event or defend event)	char(2)	XX						
	type	type of the game event (goal, card, substitution)	char(2)	XX						
	player_in_id	specific number for player that got substituted	int	x						
	player_assist_id	specific number for player that assists with the scoring	int	x						
	defend_type	type of the defend event (save, miss, others)	char(2)	XX						
	player_id	player's specific number	int	x					FK	player_id(Player)
Player_Valuation	player_id	player's specific number	int	xXXXXxx	Y Y		PK			
	date	date of the valuation	date	yyyy-mm-dd						
	valuation_id	valuation's specific number	int	x						
	market_value_in_eur	market value in euro	int	x						
Transfer	transfer_date	date of transfer	date	yyyy-mm-dd	Y Y Y		PK			
	salary	player's salary	int	x						
	transfer_id	transfer's specific number	int	x						
	player_fname	player's first name	varchar(20)	Xxxxxx						
	player_lname	player's last name	varchar(20)	Xxxxxx						
	club_to	club transferring to	varchar(50)	Xxxxxx						
	club_from	club transferring from	varchar(50)	Xxxxxx						
	release_clause	release clause amount	int	x						
	transfer_fee	transfer fee	int	x						
	transfer_season	transfer season	varchar(5)	xx/xx						
transfer_exp_date	transfer expiration date	date	yyyy-mm-dd							
Reach	valuation_id	valuation's specific number	int	x			PK,FK PK,FK PK,FK	valuation_id(Player Valuation) t_n_id(Transfer Negotiation) transfer_id(Transfer)		
	t_n_id	transfer negotiation's specific number	int	x						
	transfer_id	transfer's specific number	int	x						
Move To	transfer_id	transfer's specific number	int	x			PK,FK PK,FK PK,FK	transfer_id(Transfer) club_id(Club) player_id(Player)		
	club_id	club's specific number	int	x						
	player_id	player's specific number	int	x						
Player Rating	rating_id overall_rating	rating's specific number overall rating of a player	int decimal(10,1)	x x	Y	1-10	PK			
Calculate	rating_id	rating's specific number	int	x			PK,FK PK,FK PK,FK PK,FK	rating_id(Player Rating) criteria_id(Club Criteria) player_id(Player) club_id(Club Criteria)		
	criteria_id	criteria's specific number	int	x						
	player_id	player's specific number	int	x						
	club_id	club's specific number	int	x						

6. Database Implementation in mySQL server

Once the details of the database had been finalized (as per the previous chapter), the database was implemented in mySQL server.

6.1 Imports of the initial Kaggle dataset

The 5 initial data from Kraggle, which include: Players, Clubs, Transfers, Game Events (referred to as “Events” in our database), and Player valuation, were imported into the DBMS via the “Table Data Import Wizard” in the MySQL Workbench. To ensure the accuracy of the import data to the original files from Kraggle, most of the imported data was imported as a text data type. This is because when we were importing the data into multiple data types, we experienced multiple errors that prevented accurate data transfers.

Since the original data has empty strings (‘’) spread all over the data, they have to be processed before they can be converted to their respective data type. To do this, we use

```
UPDATE table_name
SET column_name = null
WHERE column = ‘’;
```

to set all empty strings to null. This method is used mainly to convert from a text to an int data type. For date and time data, we use

```
UPDATE table_name
SET column_name = STR_TO_DATE( str, format);
```

to convert the data to a format that MySQL can accept. For example, “01/08/2005” will be converted to “2005-08-01”. This converted data can then be converted from a string to a datetime data type. Lastly, text data were converted to the varchar data type directly without additional steps.

As the data dictionary includes more attributes that doesn’t exist in Kaggle, modifications must be made. Columns were added using **ALTER TABLE** while its values are added to suit each attribute’s nature.

The ‘players’ table lacked ‘goals’, ‘assist’, ‘save’, ‘games’. Values for these attributes are retroactively added using **UPDATE**:

- attribute ‘games’ are added by **25*(2.5 * (RAND()))** since players generally played around 30 games per season. High participation, up to 55 games is possible.

- attribute 'goal' is initially added by **FLOOR(current_club_id / player_id)**. Then, any value more than 40 is divided by 1.5 multiple times until no values are modified. This simulates the vast variousness of goals scored by players while keeping values at a reasonable level (Strikers are usually expected to score 30 goals per season. Though some players are able to score more goals.)
- attribute 'saves' is added in a similar way, although we **FLOOR(player_id/current_club_id)** instead (As goal keepers don't usually score any goals and vice versa)
- attribute 'assist' is added by **FLOOR(30 * rand()/(((goal + save)/20)+1))**. This is so that players with high goals or saves don't have high assists. Note that +1 is added to prevent dividing by zero.

The 'transfer' table lacked 'transfe_exp_date' and 'salary'. 'transfer_exp_date' is added using **ADDDATE** to add 5 years to the transfer date (a typical duration of a player's contract) with a variance between 0.6 and 1.5 times. 'salary' is added with a random value of between 1,000,000 to 200,000,000 as football players' real world earnings vary significantly.

The 'player_valuation' table doesn't have valuation_id, originally. To add the valuation_id to every entry of the table and increment the value along the table, we use:

```
ALTER TABLE player_valuation
ADD COLUMN valuation_id INT PRIMARY KEY
AUTO_INCREMENT FIRST;
```

6.2 Creation of supplementary data objects

The other 7 data objects: "Club Criteria", "Transfer Offer", "Transfer", "Negotiation", "Player Rating", "Reach", "Move to", and "Calculate" are implemented into the database using DDL. The entries were then inserted into the tables as per the attached file.

For Transfer_offer, it is not common for the transfer fee to be zero. On the other hand, it is possible that the transfer fee can be over 200 million euro but this case is very rare. The average transfer fee is 4 millions.

For Transfer_Negotiation, on average 50% of transfer offers from clubs will be accepted by a player.

For Player_Rating, it evaluate them based on their specific position using key metrics like shots on goal for forwards, assist for midfielder, tackles won for defenders, or saves for goalkeepers, often using algorithms that weight the importance of each statistic

For Club_Criteria, this table represents clubs' desired traits of a player by providing weight values to the table when looking for new players. Clubs tend to favor players that have enough endurance to play many games in a season, so game_cri should be the highest. Clubs will look for players to fill in their currently lackluster positions, which means weights of 3 other criterias should skew towards one value. As a result, entries of this table were randomly generated according to the following rule:

- all values in an entry sums up to 100
- weights of game_cri are between 50 +/- 10
- weights of other criterias are calculated successively with high degree of variance (eg. save_cri = ((100 - game_cri) * rand(3))/3)

30 rows of stand-in values are inserted before **UPDATE** is used to edit the entries.

Ternary

The 3 ternary relations: “reach”, “move_to”, “calculate” are implemented into the database using DDL. The relation implements IDs from the entities that it connects to. The entries were then inserted into the table as per the attached file.

6.3 Implementation of Primary and Foreign Keys

Once the data objects were completed, primary keys and foreign keys were created to link the data objects to match the relational schema.

To add primary keys, we used:

```
ALTER TABLE table_name  
ADD CONSTRAINT pk_name  
PRIMARY KEY(column_name);
```

To add foreign keys, we used:

```
ALTER TABLE table_name  
ADD CONSTRAINT fk_name  
FOREIGN KEY(column_fk)
```

REFERENCES parent_name(column_pk);

When we were adding a foreign key to the 'events' table referencing the 'players' table, we encountered an error. Further investigation revealed that there are multiple data points within the 'events' table that do not exist in the 'players' table. To counter this, we chose to remove these data entries from the table.

Those data were discovered with this query:

```
SELECT player_id FROM events  
WHERE player_id NOT IN  
(SELECT player_id FROM players);
```

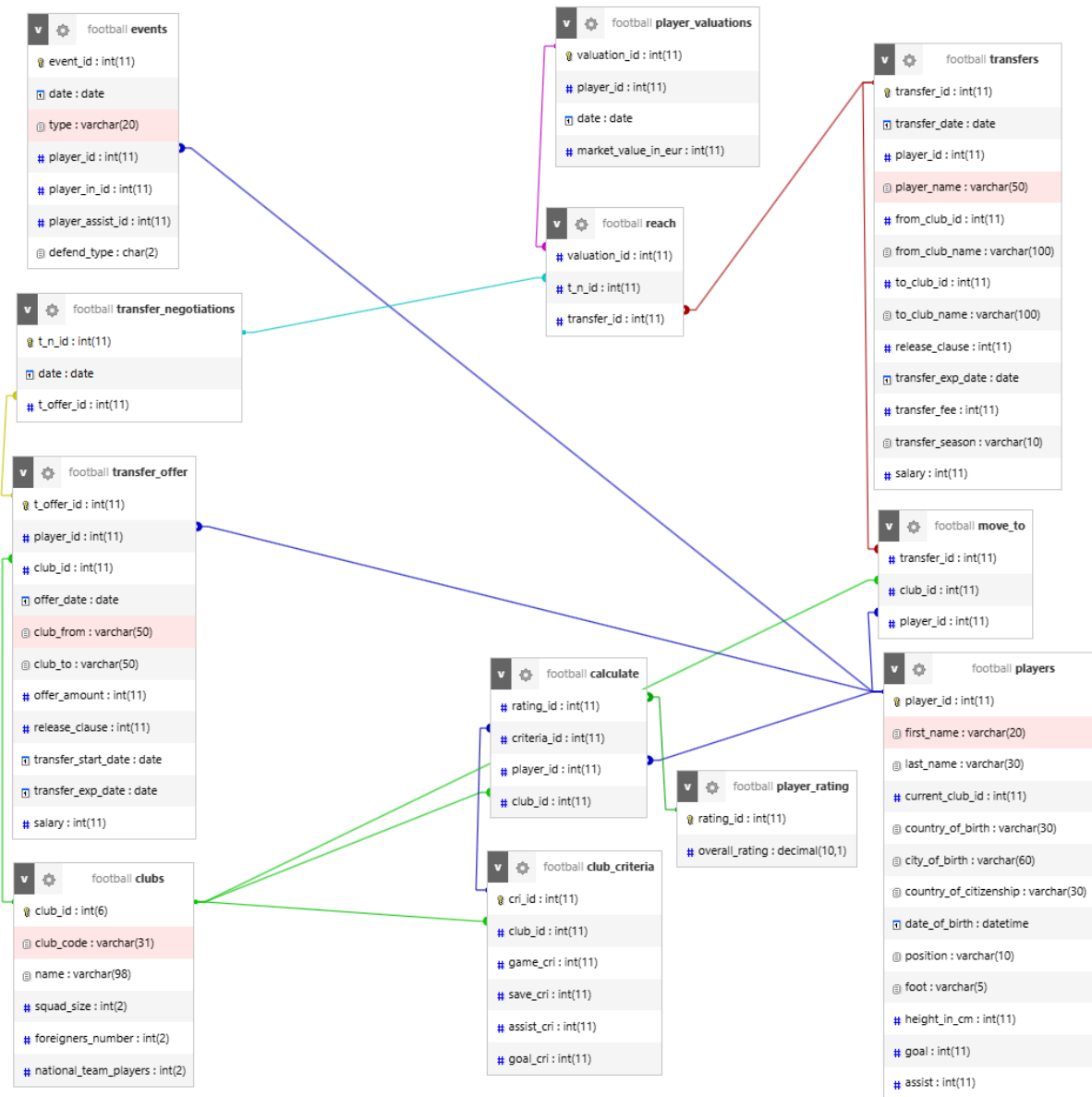
To delete this data, we first created a view of this data:

```
CREATE VIEW test AS  
SELECT player_id FROM events  
WHERE player_id NOT IN  
(SELECT player_id FROM players);
```

We delete this data from the 'events' table by:

```
DELETE FROM events e  
INNER JOIN test t  
ON e.player_id = t.player_id  
WHERE e.player_id = t.player_id;
```

After these procedures, the foreign key was added successfully, as well as the other foreign keys of other tables.



5. Links

These links provide clearer versions of all diagrams:

Revised ERD:

<https://drive.google.com/file/d/1U6b14UKpqzByAaXmAzy0kWybIQkfNQW/view?usp=sharing>

MiniERD and 8 step transformation:

<https://drive.google.com/file/d/1JcGbzMposBpsV6mL-0zWJY6XmFUclaBH/view?usp=sharing>

Data Dictionary:

<https://drive.google.com/file/d/1SwQ0OfjY5tp6NTlbmUYsVpJSE3pdkn6-/view?usp=sharing>